

Draft: A Survey of Type Theories:
Martin L f, Homotopy and Cubical
A Short Review

Naveen Maurya

April 14, 2026

1 Introduction

Type theory is a family of foundational systems, many based on a dependently typed λ -calculus. The Curry-Howard correspondence links proofs and programs and allows theorem provers like Coq, Agda, Scala, Lean etc. powered by type theories, to assist in machine-checking proofs in a constructive and computational framework. [3] In this report, we shall describe Martin-Löf, Homotopy, and Cubical Type Theories.

2 Basic Description

We shall describe a standard version of type theory called (intensional) Martin-Löf Type Theory (MLTT). Set theoretic foundations usually consist of a number of *axioms*, written in the language of first-order logic, which is separately assumed beforehand. The logic's deductive system dictates how inferences can be made (how the logic behaves), while the *axioms* only define and describe sets and how they behave. This forms a basis to build mathematics, using sets to describe mathematical objects. However, MLTT combines both of these together into the deductive system itself. The deductive system consists of the primitive notion of 'types', the building blocks for mathematical objects here, and specifies by itself how they ought to behave.

MLTT consists of *rules* for deriving *judgements*, which are either of the form $a : A$ meaning ' a is a term/instance of type A ', or $a \equiv b : A$ meaning ' a is judgementally equal to b ', while both being instances of A . Judgemental equality is a strict notion of equality which classifies objects which are definitionally the same within the intrinsic language of the theory. A separate notion 'propositional equality', denoted with $=$, is a weaker but more faithful to the original notion of equality of mathematical objects.

The language of MLTT is an extension of dependently-typed λ -calculus, with variables $x_1, x_2, \dots$, primitive constants $c, c', c'', \dots$, and defined constants f, f', f'' . The terms $t, t', t'', \dots$ are given by the grammar

$$t \rightarrow x \mid \lambda x. t \mid t(t') \mid c \mid f$$

The logic of MLTT comes from treating propositions as types, wherein having an instance/term of a 'proposition as a type' is interpreted as a witness/evidence/proof of the proposition. Other semantics of Type Theory to be described also afford further interpretation under this lens. [5]

2.1 Universes

Type Theory 'postulates' a chain of cumulative universes à la Russell, $\mathcal{U}_0, \mathcal{U}_1, \mathcal{U}_2, \dots$ as primitive constants with

- $\mathcal{U}_i : \mathcal{U}_{i+1}$ for each i
- If $A : \mathcal{U}_i$, then $A : \mathcal{U}_{i+1}$

Any type in the theory is an instance of some universe in this chain. Each universe is also a type, and an instance of universes above it in this hierarchy. But unless necessary, we omit the index from the universe and write $A : \mathcal{U}$.

Remark. This chain, while indexed by natural numbers, is not related to the Natural Numbers that will be defined in the theory. The indices are come the metatheory. We shall see, the arithmetic object of Natural Numbers ($\mathbb{N}$) is richer and needs more information to be properly defined.

3 Induction

The rest of the deductive rules of Type Theory go on to give schemas to construct, manipulate, and eliminate various constructions (known as type formers/inductive types). A number of these constructions are *synthetically* baked into the deductive system, known as *type formers*. Type Theories also allow the user to define their own constructions, known as inductive types, requiring the same criteria to be defined.

The general scheme for these type formers/inductive types is as follows. An type former/inductive type is defined by:

- A list of **formation rules**, describing how the inductive type can be introduced into context.
- A list of **constructors/introduction rules**, to introduce new terms of these types.
- **Elimination rules** to give a recipe how to use/consume elements of the type.
(Usually, this involves a *recursion principle* to define functions on the type, and a stronger *induction principle* to define dependent functions on the type.)
- A **computation rule**, that tells how the elimination acts on the constructors.
- An (optional) **uniqueness principle**, dictating how each element of the type is uniquely determined by the result of elimination rules applied to it.

Here is a brief, informal description of these type formers in Type Theory. Each of them also have a logical interpretation to them under 'propositions as types'. A detailed, formal description is available in Appendix I of [6].

3.1 Some Type Formers/Inductives

3.1.1 Function Types

Given $A, B : \mathcal{U}$, we can form $A \rightarrow B : \mathcal{U}$, the type of functions from A to B , syntactically given by a simple λ -calculus. Introducing a function can be done by specifying a formula as $(\lambda x. \Phi)$ where Φ is a formula in the variable x . Elimination is performed by evaluation, computed as $(\lambda x. \Phi)(a) \equiv \Phi[a/x]$, replacing all occurrences of x with a . We get $f(a) : B$ for any $f : A \rightarrow B, a : A$.

Iterated function types such as $A \rightarrow (B \rightarrow C)$ are effectively multivariable function types (this is called currying, due to Product-Exponential adjunction), so the parentheses are omitted and associated by right from default, like $A \rightarrow B \rightarrow C$. Currying is also used in lambda notation, writing $(\lambda x. \lambda y. \Phi)$ instead of $(\lambda x. (\lambda y. \Phi))$.

Logically, $A \rightarrow B$ corresponds to $A \implies B$.

3.1.2 Dependent Function Types (Π -types)

Given $A : \mathcal{U}, B : A \rightarrow \mathcal{U}$, the type of dependent functions is $\prod_{(x:A)} B(x)$, functions with codomain dependent on the domain. They follow exactly the same rules as regular function types, but $f(a) : B(a)$ for $a : A, f : \prod_{(a:A)} B(a)$.

Logically, Π -types correspond to for-all predicates ($\forall$).

3.1.3 Product Types

Given $A, B : \mathcal{U}$, the product type $A \times B : \mathcal{U}$ consists of pairs (a, b) , given by $a : A, b : B$. Let $E : A \times B \rightarrow \mathcal{U}$, then to define $f : \prod_{(z:A \times B)} E(z)$, it suffices to define f

for the special case where $x \equiv (a, b)$ for some $a : A, b : B$. This is the *induction principle*, denoted $\text{ind}_{A \times B}$. The computation rule enforces the defined function matches in the special case. Formally, this is

$$\text{ind}_{A \times B} : \prod_{(E : A \times B \rightarrow \mathcal{U})} \left(\prod_{(x : A)} \prod_{(y : B)} E((x, y)) \rightarrow \prod_{(z : A \times B)} E(z) \right)$$

with the computation rule

$$\text{ind}_{A \times B}(E, g, (a, b)) \equiv g(a)(b)$$

But from now on we'll often condense this into informal language for convenience.

Logically, $A \times B$ corresponds to $A \wedge B$.

3.1.4 Coproduct Types

Given $A, B : \mathcal{U}$, the coproduct type $A + B$ is given by two generators $\text{inl} : A \rightarrow A + B$, and $\text{inr} : B \rightarrow A + B$. For its induction principle, to define a function $f : \prod_{(z : A + B)} E(z)$, it suffices to define f for the special case where $z \equiv \text{inl}(x)$ and the case where $z \equiv \text{inr}(y)$.

Logically, $A + B$ corresponds to $A \vee B$.

3.1.5 Dependent Pair Types (Σ -types)

This is the dependent version of a product type. Given $A : \mathcal{U}, B : A \rightarrow \mathcal{U}$, we have $\sum_{(a : A)} B(a) : \mathcal{U}$. Say $a : A, b : B(a)$, one can form $(a, b) : \sum_{(a : A)} B(a)$. Let $E : \sum_{(a : A)} B(a) \rightarrow \mathcal{U}$. To define $f : \prod_{(c : \sum_{(a : A)} B(a))} E(c)$, it suffices to define f in the special case where $c \equiv (a, b)$ for some $a : A, b : B(a)$.

Logically, Σ -types correspond to there-exists predicate ($\exists$).

3.1.6 0, 1 and 2

The empty type **0** is the type with no constructors, and the induction principle, that to define $f : \prod_{(x : \mathbf{0})} E(x)$, it suffices to *not define anything*. By default, we have a function, $\prod_{(x : \mathbf{0})} E(x)$.

The unit type **1** is the type with a single constructor, $\star : \mathbf{1}$. The induction principle says that to define $f : \prod_{(x : \mathbf{1})} E(x)$, it suffices to define f for $x \equiv \star$.

The boolean type **2** is the type with two constructors, $0_2 : \mathbf{2}$ and $1_2 : \mathbf{2}$. To define $f : \prod_{(x : \mathbf{2})} E(x)$, it suffices to define for $x \equiv 0_2$ and $x \equiv 1_2$. Alternatively, one can define $\mathbf{2} \equiv \mathbf{1} + \mathbf{1}$.

Logically, **0** is a contradiction ($\perp$) and **1** is True ($\top$).

3.1.7 Natural Numbers Type ($\mathbb{N}$)

The natural numbers type is the prototypical example of an *Inductive Type*. It is generated by the constructors

- $\text{zero} : \mathbb{N}$
- $\text{succ} : \mathbb{N} \rightarrow \mathbb{N}$

Let $E : \mathbb{N} \rightarrow \mathcal{U}$. Then to define $E(n)$ for all $n : \mathbb{N}$, it suffices to define it for the following special case $n \equiv \text{zero}$, and for all $m : \mathbb{N}$, given $f(m)$ define it for $n \equiv \text{succ}(m)$. (This is almost the usual principle of mathematical induction!)

3.1.8 Identity Types

Given $A : \mathcal{U}$, we have the identity type family $\text{Id}_A : A \rightarrow A \rightarrow \mathcal{U}$. Given $x, y : A$, we have $\text{Id}_A(x, y)$, also denoted as $x =_A y$, which is the type of all *propositional equalities* between x and y . The introduction rule is by using $\text{refl} : \prod_{(a : A)} (a =_A a)$. Identities being types themselves allows equality of objects to also be treated as points, susceptible to comparison with other equalities, and thus proofs of equalities as mathematical objects susceptible to proof of equality, ad infinitum. Thus, one obtains an infinite-order logic as opposed to set theory's first-order.

The induction principle for Identity Types is called path induction, which says given $C : \prod_{(x, y : A)} (x =_A y) \rightarrow \mathcal{U}$, a function $c : \prod_{(x : A)} C(x, x, \text{refl}_x)$, there is a function $f : \prod_{(x, y : A)} \prod_{(p : x =_A y)} C(x, y, p)$ such that $f(x, x, \text{refl}_x) \equiv c(x)$. That is, to define a function on paths between points in A , it suffices to exhibit a function on all reflexive loops refl_x over all points $x : A$.

3.2 Custom Inductive Types

Type Theories provide a schema to build newer, user-defined inductive types as well. Conceptually, an inductive type is a type *freely generated* by a number of constructors, which may involve functions that output the given type by (possibly) taking as input terms of the same type. This intuition needs to be formalized carefully so as to ensure that only meaningful inductive types may be constructed.

Remark. *Inductive types are objects one can perform induction on! Similarly to $\mathbb{N}$, the induction principle for a given inductive type is the statement that to prove a statement for all its instances, it suffices to prove it in a few special cases, built on the constructors of the type.*

3.2.1 W-Types

There is a complete formalism of inductive types up to certain demands for the constructors.

Definition 1 (W-Types). *Given $A : \mathcal{U}, B : A \rightarrow \mathcal{U}$, the W-Type $\text{W}_{(a : A)} B(a)$ is defined by the constructor*

$$\text{sup} : \left(\prod_{(a : A)} B(a) \rightarrow \text{W}_{(a : A)} B(a) \right) \rightarrow \text{W}_{(x : A)} B(x)$$

and the induction principle, to prove $E : \text{W}_{(a : A)} B(a) \rightarrow \mathcal{U}$ for all elements of $\text{W}_{(a : A)} B(a)$, it suffices to prove it for $\text{sup}(a, f)$ for each $a : A$, and $f : (B(a) \rightarrow \text{W}_{(a : A)} B(a))$, assuming it holds for all $f(b)$ with $b : B(a)$.

A W-Type $\text{W}_{(a : A)} B(a)$ may be interpreted as the type of well-founded trees with A -many ways to branch out from each node, corresponding to a constructor. Each $a : A$ corresponds to a constructor, each of which take $B(a)$ -many instances of $\text{W}_{(a : A)} B(a)$ as branches. The type $\text{W}_{(a : A)} B(a)$ is the type of objects *freely generated* by these constructors. If an inductive type may be expressed equivalently as generated by sup as described, it can be formalized as a W-Type.

- The boolean type **2** is equivalent to the W-Type $\text{W}_{(a : A)} B(a)$ where $A \equiv \mathbf{2}$ with $B(0_2) \equiv B(1_2) \equiv \mathbf{0}$, that is generated by two constructors, $0_2 : \mathbf{2}$ and $1_2 : \mathbf{2}$.

- The natural numbers type $\mathbb{N}$ is equivalent to $W_{(a:A)}B(a)$ where $A \equiv \mathbf{2}$, with $B(0_2) \equiv \mathbf{0}$ and $B(1_2) \equiv \mathbf{1}$, i.e., generated by two constructors, corresponding to $\text{zero} : \mathbb{N}$, and $\text{succ} : \mathbb{N} \rightarrow \mathbb{N}$.
- The type $W_{(x:1+A)}B(x)$ with $B(\text{inl}(\star)) \equiv \mathbf{0}$ and $B(\text{inr}(a)) \equiv \mathbf{1}$, corresponds to the type $\text{List}(A)$ of finite length strings over the alphabet A , i.e. generated by $\text{nil} : \text{List}(A)$ and $\text{cons} : A \rightarrow \text{List}(A) \rightarrow \text{List}(A)$.

4 Homotopy Type Theory

Homotopy Type Theory (HoTT) extends MLTT by introducing a topological/homotopical interpretation of types.

4.1 A Homotopical Interpretation

In HoTT, types can be interpreted as either topologically as spaces, or as ∞ -groupoids from higher-dimensional category theory. Any topological space can be classified upto homotopy equivalence by considering its *fundamental ∞ -groupoid*. This follows from the fundamental ∞ -groupoid construction being adjoint to the geometric realization functor. (This is the *homotopy hypothesis/theorem*), first talked about in Grothendieck's *Pursuing Stacks* [1].

4.2 Identity Types

The precise manner in which types correspond to spaces or ∞ -groupoids is demonstrated by constructing an associated ∞ -groupoid.

In the homotopy interpretation, the identity type $\text{Id}_A(x, y)$ corresponds to the space of paths from $x : A$ to $y : A$ endowed with the compact-open topology. refl_x is the constant path at x . The iterated path type, $\text{Id}_{\text{Id}_A(x, y)}(p, q)$ corresponds to 2-dim paths/homotopies between $p, q : x =_A y$, and so on...

Proposition 1. *Given $x, y, z : A$, there exist the following maps*

- (i) $(x = y) \rightarrow (y = x)$
- (ii) $(x = y) \rightarrow (y = z) \rightarrow (x = z)$

Proof. (i) By path induction, assume $x \equiv y$ and $p \equiv \text{refl}_x$. Now simply exhibit $\text{id}_{x=A} \equiv (\lambda a.a)$. (ii) By path induction, assume $x \equiv y$ and $p \equiv \text{refl}_x$. Exhibit $(\lambda a.\lambda b.b)$. $\square$

Denote the map in (i) as $(-)^{-1}$, and (ii) as $(-\cdot -)$.

Proposition 2. *Let $x, y, z, w : A$, $p : x =_A y, q : y =_A z$ and $r : z =_A w$. Then we have:*

- (i) $p = \text{refl}_x \cdot p$, and $p = p \cdot \text{refl}_y$
- (ii) $p \cdot p^{-1} = \text{refl}_x$ and $p^{-1} \cdot p = \text{refl}_y$
- (iii) $(p \cdot q) \cdot r = p \cdot (q \cdot r)$

Proof. (i) By path induction, assume $x \equiv y$ and $p \equiv \text{refl}_x$. Now exhibit $\text{refl}_{\text{refl}_x}$ or $\text{refl}_{\text{refl}_y}$. (ii) By path induction, assume $x \equiv y$ and $p \equiv \text{refl}_x$. Exhibit $\text{refl}_{\text{refl}_x}$ or $\text{refl}_{\text{refl}_y}$. (iii) By path induction, assume $y \equiv z$ and $q \equiv \text{refl}_y$. Exhibit refl_p to obtain $(p \cdot q) \equiv p$, and similarly refl_r to obtain $(q \cdot r) \equiv r$. Our goal reduces to $p \cdot r = p \cdot r$. Exhibit $\text{refl}_{p \cdot r} : p \cdot r = p \cdot r$. $\square$

A remarkable feature is that while (i) and (ii) can be proven judgementally, (iii) is only true propositionally, i.e., upto a path in the higher path space. Strikingly, we also have all the required higher coherences propositionally us-

ing only path induction. This gives us the (weak) ∞ -groupoid interpretation which is made functorial by the following result.

Proposition 3. *Let $f : A \rightarrow B, g : \prod_{(a:A)} P(a), x, y : A$, and $p : x =_A y$, then we have the following:*

- (i) (Application) $\text{ap}_f(p) : (f(x) =_B f(y))$
- (ii) (Transport) $p_* : P(x) \rightarrow P(y)$
- (iii) (Dependent Application) $\text{apd}_g(p) : (p_*(g(x)) =_{P(y)} g(y))$

Proof. For all, use path induction. Assume $x \equiv y$ and $p \equiv \text{refl}_x$ for all. (i) Exhibit $\text{refl}_{f(x)}$. (ii) Exhibit $\text{id}_{P(x)} \equiv (\lambda z.z)$. (iii) Exhibit $\text{refl}_{g(x)}$. $\square$

4.3 Axioms in HoTT

Some expected semantic behaviour does not directly follow from the machinery of HoTT laid out so far. To enforce such behaviour we introduce a few ‘axioms’ into our theory, which are merely inhabitants of some type without an inductive construction. But first, we make some definitions.

Definition 2 (Homotopy of maps). *Let $f, g : \prod_{(x:A)} B(x)$, then we say f is homotopic to g , denoted*

$$(f \sim g) \equiv \prod_{(x:A)} (f(x) =_{B(x)} g(x))$$

Definition 3 (Equivalence). *Let $A, B : \mathcal{U}$. We say A and B are (homotopy) equivalent if there is an inhabitant of*

$$(A \simeq B) \equiv \sum_{(f:A \rightarrow B)} \sum_{(g:B \rightarrow A)} (g \circ f \sim \text{id}_A) \times (f \circ g \sim \text{id}_B)$$

The notion of equality for functions is too restrictive to capture their expected behaviour. Classically, we expect two functions to be equal if they take the same values on all the inputs. To ensure semantic well-behavedness, we must enforce this as an axiom.

Axiom 1 (Function Extensionality). *Let $A : \mathcal{U}, B : A \rightarrow \mathcal{U}$ and $f, g : \prod_{(x:A)} B(x)$, then we have $(f = g) \simeq (f \sim g)$.*

That is, equality of maps is characterized entirely by their values in the codomain. In particular, if two functions are homotopic, they are equal.

In [1], Voevodsky proposed the univalence axiom, which is inspired from the mathematical practice of identifying isomorphic objects.

Axiom 2 (Univalence). *For $A, B : \mathcal{U}$, there is an equivalence: $(A =_{\mathcal{U}} B) \simeq (A \simeq B)$*

In particular, if $A \simeq B$, then $A =_{\mathcal{U}} B$. Roughly speaking, this means “isomorphic objects are equal”. In a univalent universe $\mathcal{U}$, isomorphic objects/equivalent types can be treated as equal and one can transport (theorems/proofs/properties) across them.

4.4 Higher Inductive Types

We can now expand to types generated by constructors along with relations. We let the constructors possibly be higher-dimensional paths between points. This way, free generation is equivalent to generation on points quotiented by relations (corresponding to these higher paths).

Such a *higher inductive type* hardcoded in HoTT is the circle type $\mathbb{S}^1$ generated by the constructors

- A point $\text{base} : \mathbb{S}^1$
- A path $\text{loop} : \text{base} =_{\mathbb{S}^1} \text{base}$

This corresponds to a chosen basepoint base , and a non-trivial path loop connected the basepoint, giving a circle.

Many familiar mathematical objects such as the integers, CW-complexes etc. can be built using this schema. Various approaches exist to formalize such a schema, but have only yielded partial satisfaction. Some of the existing approaches are using Quotient Inductive Types [], Homotopy Initial Algebras [], and computationally most satisfactory, Cubical Type Theory []. A completely satisfactory formalism remains an open problem.

5 Cubical Type Theory

HoTT fails to achieve desirable computational behaviour. In particular, Univalence is an axiom and Canonicity is not guaranteed for HITs. HoTT affords presheaf model in simplicial sets. A computational model of type theory is then motivated using cubical sets in place for simplicial sets.

5.1 Cubical Sets

Let there be a fixed discrete and countable set of *names* $\mathbb{A}$ and define the set of *directions* $\mathbf{2} = \{0, 1\}$.

Definition 4 (The cubical category $\square$). *[Coq] The cubical category $\square$ has as objects finite subsets $I, J, K \dots \subseteq \mathbb{A}$. A morphism $f : I \rightarrow J$ is a set function $f : I \rightarrow J \cup \mathbf{2}$, such that $fx = fy$ implies $x = y$ for $x, y \notin \mathbf{2}$. The composition of morphisms $f : I \rightarrow J$ and $g : J \rightarrow K$ is defined as $g \circ f(x) = g(f(x))$ when $fx \notin \mathbf{2}$ and fx otherwise.*

We will call morphisms *substitutions* as they encode sending a name to a direction. We will also denote fg for $g \circ f$ as well as xf for $f(x)$ from now on (called diagram order), and often omit braces in $I \cup \{x, y\}$ or $I - \{x, y\}$ for finite sets of names.

For $I \subseteq J$, we call the inclusion $f : I \rightarrow J$ a *degeneracy map*. We have the *face maps* $(x = 0), (x = 1) : I \rightarrow I - x$ sending x to 0 and 1 respectively. (Note how these constructions parallel the simplex category Δ)

Definition 5 (Cubical sets). *A cubical set X is a covariant functor $X : \square \rightarrow \mathbf{Set}$, i.e., a presheaf over $\square^{op}$.*

That is, a cubical set X is a collection of sets $X(I)$ and for any substitution $f : I \rightarrow J$, set functions $X(I) \rightarrow X(J)$, $u \mapsto uf$ such that $u\mathbf{1} = u$ and $u(fg) = (uf)g$. This can be thought of as $X(\emptyset)$ is the set of points of a space, $X(x)$ are its lines, $X(x, y)$ are squares, and so on.

$$\begin{array}{ccc} u(0, 1) & \xrightarrow{u(y=1)} & u(1, 1) \\ \uparrow u(x=0) & & \uparrow u(x=1) \\ u(0, 0) & \xrightarrow{u(y=0)} & u(1, 0) \end{array} \quad \begin{array}{c} y \\ \downarrow \\ x \end{array}$$

Cubical sets form a presheaf category, i.e. $\mathbf{cSet} := [\square, \mathbf{Set}]$ with morphisms given by natural transformations.

Example 5.1. *Let R be a commutative ring. Then $R[\cdot]$ is a cubical set, with $R[I]$ being the polynomial ring over*

I , and for $f : I \rightarrow J$, $R[f](x) = f(x)$ where 0 and 1 in $\mathbf{2}$ are swapped for 0 and 1 from R .

Example 5.2 (Interval). *The interval $\mathbb{I}$ is the cubical set defined by $\mathbb{I}(I) = I \cup \mathbf{2}$ and for $f : I \rightarrow J$, $\mathbb{I}(f)$ is the extension of the set theoretic map of f such that $\mathbb{I}(f)(0) = 0$ and $\mathbb{I}(f)(1) = 1$.*

Note that for a fixed name x , $\mathbb{I} \cong \mathbf{y}\{x\}$, where $\mathbf{y}$ is the Yoneda embedding. More generally, $\square_I = \mathbf{y}I$ where I contains n elements, is the representable n -cube (since $\mathbf{y}I \cong \mathbf{y}J$ for $I \cong J$).

Definition 6 (Non-dependent Path Space). *Given a cubical set X , we define its path space $\mathbb{I}X$ as a cubical set $(\mathbb{I}X)(I) = X(I, x_I)$ where x_I is a fresh name ($x_I \notin I$). Let $f : I \rightarrow J$ be a substitution, then $X(f)$ is given by the induced map from f via inclusion, composed with substituting x_I for x_J , i.e., $\omega(X(f)) = \omega f(x_I = x_J)$.*

For a cubical set X , $\mathbb{I}X$ is also a cubical set. $\mathbb{I}X(\emptyset)$ are the points, $\mathbb{I}X(x)$ are the lines, $\mathbb{I}X(x, y)$ are the squares, and so on.

Definition 7 (Non-dependent Identity Type). *Given a cubical set X , and $a, b \in X(\emptyset)$, the identity type $\text{Id}_X(a, b)$ is the subobject of $\mathbb{I}X$ such that $\omega \in (\text{Id}_X(a, b))(I)$ iff there is some name $x \in I$ such that $\omega(x = 0) = a$ and $\omega(x = 1) = b$.*

5.2 The Uniform Kan Condition

Definition 8 (Open boxes). *Let X be a cubical set. Let I be a finite set of names, and $x, J \subseteq I$, with $x \notin J$. Fix a either 0 or 1. An open box denoted $\vec{u}$ is a family of faces u_{yb} in $X(I - y)$ for each (y, b) either (x, a) or $y \in J, b = 0, 1$. such that*

$$u_{yb}(z = a) = u_{za}(y = b)$$

for all $y, z \in J, y \neq z$ and $b, c = 0, 1$.

The idea is that an open box contains all faces, except for one face in a direction x .

Definition 9 (Kan Cubical Set). *A Kan cubical set X is a cubical set X , equipped with filling operations $X\uparrow$, and $X\downarrow$, such that for every $J, x \in I$, $X\uparrow\vec{u}, X\downarrow\vec{u}$ are in $X(I)$, where $\vec{u}$ is an open box with u_{x0} and u_{x1} in $X(I - x)$ in the respective cases $X\uparrow\vec{u}$ and $X\downarrow\vec{u}$, such that the following condition holds:*

$$(X\uparrow\vec{u})(y = b) = u_{yb} \text{ and } (X\downarrow\vec{u})(x = b) = u_{yb}$$

along with the ‘uniformity/coherence’ condition, that for $f : I \rightarrow K$ defined on J, x , we require:

$$(X\uparrow\vec{u})f = X\uparrow(\vec{u}f) \text{ and } (X\downarrow\vec{u}) = X\downarrow(\vec{u}f)$$

We also denote $X^+\vec{u} = X\uparrow\vec{u}(x = 1)$ and $X^-\vec{u} = X\downarrow\vec{u}(x = 0)$. This condition for X , known as the *Kan condition* is the analogous statement to “every horn has a filler” for Kan complexes, i.e., “every open box has a filler”.

$$\begin{array}{ccc} & X^+(u_{x0}, u_{y0}, u_{y1}) & \\ \uparrow u_{y0} & \xrightarrow{X\uparrow(u_{x0}, u_{y0}, u_{y1})} & \uparrow u_{y1} \\ & \xrightarrow{u_{x0}} & \end{array}$$

Any presheaf category gives rise to a model of dependent type theory by interpreting each presheaf as a type. [2] Hence, cubical sets form such a model with (dependent) products, (dependent) sums, inductive types and identity types. We refine this model by restricting to Kan Cubical Sets taken along with a specified ‘Kan Structure’, which are the Kan filling operations $X\uparrow$ and $X\downarrow$. This is needed in order to justify the elimination rules for identity types. For a complete treatment, see Simon Huber’s PhD Thesis for more details. [4]

References

- [1] Alexander Grothendieck. Pursuing stacks. *arXiv preprint arXiv:2111.01000*, 2021.
- [2] Martin Hofmann. Syntax and semantics of dependent types. In *Extensional Constructs in Intensional Type Theory*, pages 13–54. Springer, 1997.
- [3] William A Howard. The formulae-as-types notion of construction. 1995.
- [4] Simon Huber. *A model of type theory in cubical sets*. PhD thesis, Chalmers University of Technology, 2015.
- [5] Per Martin-Löf. An intuitionistic theory of types: Predicative part. In *Studies in Logic and the Foundations of Mathematics*, volume 80, pages 73–118. Elsevier, 1975.
- [6] The Univalent Foundations Program. *Homotopy type theory: Univalent foundations of mathematics*. The Univalent Foundations Program, 2013.